Activity B.1. Adding and Subtracting Objects

Activity B.1. Adding and Subtracting Objects

Print this page

OpenSCAD is a constructive-geometry CAD program. What that means is that a user defines an object and then modifies it in various ways, like scaling it, rotating, and so on. In this activity we are going to look at ways to merge objects together, or to use one object as a shaped hole in another. We can also save just the intersection of two objects.

As we saw in the "Making 2D Shapes in OpenSCAD" section of this guide, the fundamental structure of a model is an object, possibly modified. Let's look at this rotated rectangle again, and deconstruct it a little.

```
rotate([0, 0, 30]) square([15, 20]);
```

Note that we have a set of square brackets nested inside a set of parentheses. The square brackets tell us that we have a set of numbers, like the angles of rotation in three axes. The parentheses enclose however many sets of numbers we have for that particular object or modifier. If we have

```
square([15, 20], center = true);
```

we can see that the dimensions of the square and the optional item that it is supposed to be centered at the origin are both inside the parentheses.

We learned in the 2D set of activities that objects are modified starting from the closest modifier to the farthest, or right to left. In this case:

```
scale([1, 2, 0]) rotate([0, 0, 45]) square([15, 20], center = true);
```

our square was rotated and then scaled. Sometimes it does not make a difference, but usually it does.

Adding Shapes

Suppose we wanted to create an "L" shape with two rectangles. We could do that two different ways: by either adding two rectangles together, or by subtracting a smaller rectangle from a larger one. Adding two objects is accomplished by using the union() operator, which looks like this:

```
union() {
  square([15, 45]);
  square([45, 15]);
}
```

We enclose the two things we want to add together inside curly braces. By convention people indent what is inside the braces, plus the second brace, to make it easier to read.

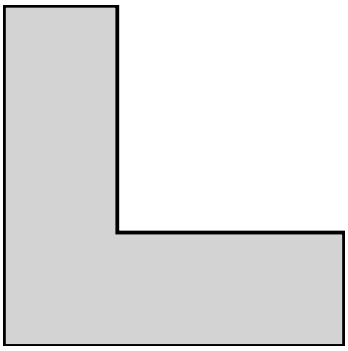

Figure 1: L-shaped figure, file union.svg

L-shaped figure, file union.svg

Subtracting Shapes

We can create the identical figure by subtracting using the difference() operator on a 15 by 15 square from the upper right corner of a 45 by 45 square, like this:

```
difference() {
  square([45, 45]);
  translate([15, 15]) square([30, 30]);
}
```

Notice that we had to translate the square before differencing. The second thing inside the difference curly braces is the thing you subtract, just like the thing after the minus sign is the thing you subtract. We created a file *difference.svg*, and if you create it you will find that it is the same figure as *union.svg*.

Intersecting Shapes

Finally, we can intersect two objects. If we intersected our 45 by 45 square with our 15 by 15 one, we would just get the smaller square since they overlap completely. If we translate the square a bit more, that will no longer be the case. Try it and check out how the results vary. Of course, if you translate the smaller square far enough, the intersection will just be nothing at all. (File *intersection.svg* shows this square.)

```
intersection() {
  square([45, 45]);
  translate([15, 15]) square([30, 30]);
}
```

Other OpenSCAD Conventions

Extra spaces and carriage returns are optional in OpenSCAD. You can put more code on one line, but it can make your model hard to read and understand if you are looking at it visually. The previous set of shapes would look like this all on one line:

```
difference() { square([45, 45]); translate([15, 15]) square([30, 30]); }
```

One place you cannot add spaces is inside one of the names of shapes ("sq uare" is not legal). OpenSCAD is also case-sensitive, so you need to be careful not to type Difference or DIFFERENCE, which will not work.

If you are creating a model that is getting a little complicated, you can also leave notes to yourself, called "comments." If we start a line with two forward slashes, like this:

```
// This is a comment
```

OpenSCAD ignores everything up to the next carriage return. If you have a longer block of text covering multiple lines you can start your long block with /* and end it with */ and OpenSCAD will ignore the whole thing. This can be handy if you want to check just part of your model.

```
/* This is also a comment

Just a longer one

*/
```

More Complex Shapes

There is no limit to how many things you can add, subtract or intersect. To do that, we can nest one or more activities inside another. You can also translate, rotate or mirror a union, difference, or intersection. For example, the following code creates a circle with an L-shaped bit taken out of its upper right hand side.

```
difference() {
  circle(50);
  translate ([15, 15]) union() {
    square([15, 45]);
    square([45, 15]);
  }
}
```

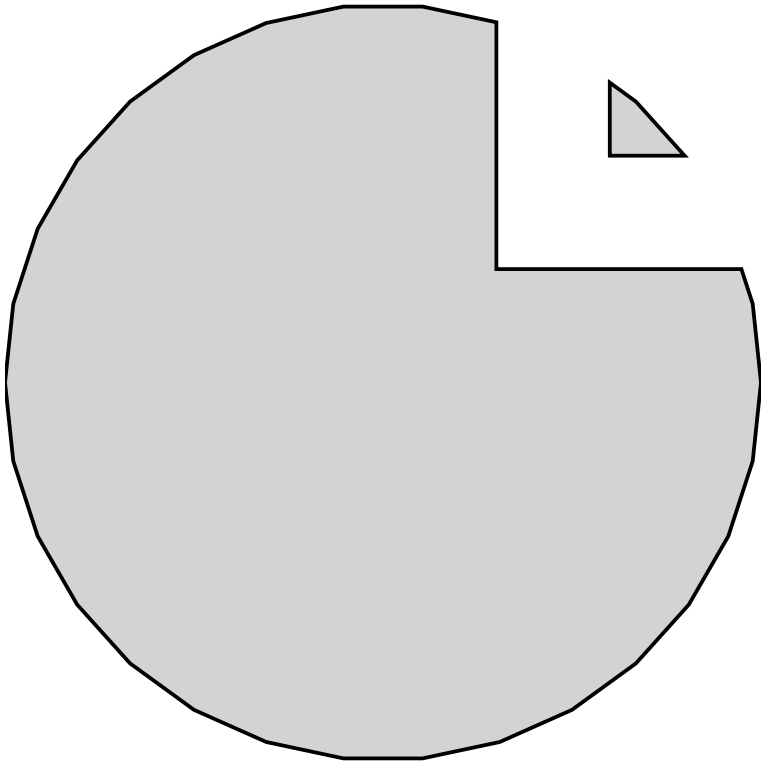

Figure 2: A circle with an L-shaped bite out of the upper right, file complexdifference.svg

A circle with an L-shaped bite out of the upper right, file complexdifference.svg

Teaching Tip: Practice

Use hands-on manipulatives, like foam shapes or Legos, to physically demonstrate union, difference and intersection before jumping into code. This tactile comparison helps students connect the code to spatial thinking and reinforces vocabulary like "subtract" or "intersect".

OpenSCAD has other capabilities like being able to create repeated sets of objects. To do that, we need a programming construct called a loop. Let's create a set of five circles marching across our page. We enclose the thing we want to do multiple times in curly brackets, just as we did for other manipulations.

Next we set up our loop, which starts with the word "for" and shows us how many times we want to do something, a little indirectly. Our loop uses a "loop counter" (the letter "i" in the example that follows) The loop runs a total of five times (starting at zero and ending at four), and it looks like this:

```
for(i = [0:1:4]) {
  translate([i * 10, 0]) circle(5);
}
```

The first time i = 0, and we use that value to see how much to translate (i times zero, or zero). The next time we translate one times 10, (10 mm) and so on. When we have gone through the fifth time, i = 4, and we translate 40 mm to draw our last 5 mm circle. You can leave out the center number (how much to step) if you are stepping by 1 each time, but we included it here for completeness.

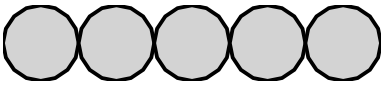

Figure 3: Five circles in a line, touching each other, file forloop.svg

Five circles in a line, touching each other, file forloop.svg

Note that an asterisk (*) means "multiply" in OpenSCAD, as it does in many other programming languages.

There are many other functions and capabilities that you can learn more about in the OpenSCAD User Manual. We will introduce a few more as we now move into 3D shapes.

Variables and Conditionals

Like most programming languages, OpenSCAD lets you assign values to named variables. This lets you assign value once, then use it in multiple places. If you later decide you want to change that value, you only need to change it in one

place. Setting a variable is done by typing the name of the variable, an equal sign, then the value you want to assign to it. This assignment is directional, so unlike in math, a = b does not mean the same thing as b = a. So for instance:

```
radius_inner = 10;
```

Would let us make a bunch of objects that all need a radius variable. One quirk of OpenSCAD relative to other programming languages is that variables can only be set once in OpenSCAD. See the OpenSCAD manual page on general syntax for more.

A variable may be assigned a value that is a number (for programmers a "floating point" number), a string (a series of characters, usually used for a word or phrase), a range (see the for loop example above), or a vector (a series of values stored in one variable, also known as an array, most commonly used for [x, y] or [x, y, z] coordinates).

OpenSCAD also has conditional statements (if … else) statements, but the rules about variables being assigned once mean that you cannot do some things that would be common in other programming languages. An if statement starts with the word "if", followed by a test condition in parentheses. What follows those parentheses will only be executed if the condition evaluates as true. This is optionally followed by an "else" statement, which is executed if the condition was false. For example:

```
a = 3;
if (a > 2) circle(5);
else square(10, center = true);
```

This will create a circle, since a = 3 which is greater than 2. If a was equal to 1, we would get a square. In other programming languages you might see something like the following, which will result in an error in OpenSCAD:

```
a = 3;
if (a > 2) b = 5;
else b = 10;
```

The same thing can be accomplished using a construction called the "ternary operator". This operator exists in most programming languages, but is rarely used in most of them.

```
a = 3;
b = (a > 2) ? 5 : 10;
```

This can be read as: b equals 5 if a is greater than 2, else b will equal 10. More information about how to use the ternary operator in OpenSCAD can be found in the OpenSCAD Documentation.

Math Functions

OpenSCAD comes with built-in math functions, including taking a square root (sqrt(4) equals 2, for example), raising a number to a power (pow(2, 3) will return the third power of 2, or 8). For example to create a circle whose radius is the square root of 2, we would write

```
circle(sqrt(2));
```

You can find the full list, and the relevant parameters, on the OpenSCAD "Mathematical Functions" manual page.

> **Note:** the images above are embedded from the original site with empty alt text — worth writing real descriptive alt text when you migrate the actual image files over.

Previous: Activity A.3: Rotating, Translating, Scaling · Download the whole thing · **Next:** Activity B.2: Moving from 2D to 3D · **Back to:** Adventure Map